

SISTEM PEMESANAN TIKET EVENT



Nama	I Gusti Bagus Anggas
NIM	2410511080
Kelas	B
Dosen	Muhammad Panji Muslim, S.Pd., M.Kom
Semester	Genap TA. 2025/2026

**UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAKARTA
FAKULTAS ILMU KOMPUTER
S1 INFORMATIKA
2026**

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Perkembangan industri hiburan dan event di Indonesia menunjukkan pertumbuhan yang signifikan. Pengelolaan tiket event secara manual atau dengan sistem monolitik konvensional seringkali menghadapi kendala seperti bottleneck performa, kesulitan scaling, dan risiko kegagalan sistem secara menyeluruh ketika terjadi lonjakan permintaan.

Untuk mengatasi tantangan tersebut, pendekatan Service-Oriented Architecture (SOA) dengan implementasi microservice menjadi solusi yang relevan. Setiap domain bisnis dipisahkan menjadi service independen yang dapat dikembangkan, dideploy, dan di-scale secara terpisah.

Laporan ini mendokumentasikan pembangunan Sistem Pemesanan Tiket Event berbasis microservice yang terdiri dari auth-service, ticket-service, payment-service, dan API Gateway sebagai single entry point.

1.2 Studi Kasus

Studi kasus yang dikerjakan adalah Sistem Pemesanan Tiket Event (digit terakhir NIM: 0) dengan ruang lingkup sebagai berikut:

- Daftar event dengan informasi lengkap (nama, lokasi, tanggal)
- Pemilihan kategori tiket per event (VIP, Regular, Student) dengan harga dan kuota
- Proses checkout dan konfirmasi pembayaran
- Generate e-ticket dengan QR code unik per tiket
- Validasi tiket di pintu masuk menggunakan QR code
- Autentikasi dengan JWT dan Google OAuth 2.0 (NIM genap)

1.3 Tujuan

- Membangun sistem berbasis microservice dengan minimal 3 service independen dan 1 API Gateway
- Mengimplementasikan REST API dengan standar HTTP method dan response code yang benar
- Membangun service menggunakan PHP MVC (Laravel 11) sebagai salah satu service
- Mengimplementasikan autentikasi JWT dengan access token dan refresh token
- Mengimplementasikan otorisasi OAuth 2.0 dengan Google sebagai provider
- Mengimplementasikan API Gateway sebagai single entry point dengan JWT validation dan rate limiting

BAB 2 ARSITEKTUR SISTEM

2.1 Gambaran Umum Arsitektur

Sistem Pemesanan Tiket Event dibangun menggunakan arsitektur microservice yang terdiri dari empat komponen utama:

Service	Teknologi	Database	Port	Fungsi
API Gateway	Node.js + Express	—	8000	Single entry point
auth-service	Node.js + Express	MySQL (auth_db)	3001	Autentikasi & otorisasi
ticket-service	Laravel 11 (PHP MVC)	MySQL (ticket_db)	3002	Manajemen event & tiket
payment-service	Node.js + Express	MongoDB (payment_db)	3003	Checkout & pembayaran

2.2 Justifikasi Pemisahan Service vs Monolitik

Pendekatan microservice dipilih dibandingkan monolitik dengan justifikasi sebagai berikut:

2.2.1 Alasan Pemisahan auth-service

Autentikasi adalah cross-cutting concern yang dibutuhkan oleh seluruh service. Dengan memisahkan auth-service, validasi JWT dapat dilakukan secara terpusat di API Gateway tanpa duplikasi logika di setiap service. Pada pendekatan monolitik, perubahan mekanisme autentikasi akan mempengaruhi seluruh modul aplikasi.

2.2.2 Alasan Pemisahan ticket-service

Domain bisnis tiket (event, kategori tiket, validasi QR) memiliki kompleksitas dan model data tersendiri dengan 4 tabel ter-relasi. Service ini wajib dibangun menggunakan PHP dengan pola MVC (Laravel 11) sesuai requirement. Pemisahan memungkinkan service ini di-scale secara independen saat terjadi lonjakan permintaan tiket tanpa mempengaruhi service lain.

2.2.3 Alasan Pemisahan payment-service

Domain pembayaran memiliki karakteristik yang berbeda — membutuhkan fleksibilitas skema data (MongoDB) untuk menyimpan informasi order yang dapat berubah strukturnya. Pemisahan juga memudahkan integrasi dengan payment gateway eksternal di masa depan tanpa mengganggu service lain.

2.2.4 Perbandingan dengan Monolitik

Aspek	Monolitik	Microservice (Sistem Ini)
Scaling	Scale seluruh aplikasi	Scale per service
Deployment	Deploy ulang seluruhnya	Deploy independen
Teknologi	Satu stack teknologi	Bebas per service
Database	Satu database bersama	Database terpisah
Fault isolation	Satu titik gagal	Terisolasi per service

BAB 3 IMPLEMENTASI

3.1 REST API + Database

3.1.1 Struktur Database ticket-service (MySQL)

ticket-service menggunakan database relasional MySQL dengan 4 tabel ter-relasi:

- Events: menyimpan data event (id UUID, name, description, location, event_date, status, created_by)
- ticket_categories: kategori tiket per event (id UUID, event_id FK, name, price, quota, sold)
- Tickets: tiket yang telah dibeli (id UUID, ticket_category_id FK, order_id, user_id, qr_code UNIQUE, status, used_at)
- Ticket_validations: riwayat validasi QR di pintu masuk (id UUID, ticket_id FK, validated_by, result, gate)

Relasi antar tabel: events → ticket_categories → tickets → ticket_validations, dengan ON DELETE CASCADE pada seluruh foreign key.

3.1.2 Paging dan Filtering

Endpoint GET /api/events mengimplementasikan paging dengan parameter page dan per_page, serta filtering berdasarkan status dan search (nama event). Contoh:

GET /api/events?page=1&per_page=10&status=active&search=konser

3.1.3 HTTP Method dan Response Code

Method	Endpoint	Response Sukses	Response Error
POST	/api/auth/register	201 Created	409, 422
POST	/api/auth/login	200 OK	401, 422
GET	/api/events	200 OK	—
POST	/api/events	201 Created	401, 403, 422
PUT	/api/events/:id	200 OK	401, 403, 404
DELETE	/api/events/:id	204 No Content	401, 403, 404
POST	/api/orders	201 Created	401, 422
POST	/api/auth/logout	204 No Content	401

3.2 REST API PHP MVC (Laravel 11)

ticket-service dibangun menggunakan Laravel 11 dengan pola MVC yang jelas:

- Model: Event, TicketCategory, Ticket, TicketValidation — menggunakan Eloquent ORM dengan trait HasUuids dan relasi Eloquent (hasMany, belongsTo)
- Controller: EventController, TicketController, TicketCategoryController — berisi logika bisnis dan validasi input via \$request->validate()
- Middleware: JwtMiddleware, RoleMiddleware — memverifikasi JWT dan role user sebelum request diteruskan ke controller
- Routes: routes/api.php — mendefinisikan semua endpoint dengan middleware yang sesuai

Service ini dikonsumsi oleh payment-service melalui inter-service call untuk generate tiket setelah pembayaran dikonfirmasi.

3.3 Otentikasi JWT

auth-service mengimplementasikan autentikasi berbasis JWT dengan ketentuan:

- Access token masa berlaku 15 menit (JWT_EXPIRES_IN=15m)
- Refresh token masa berlaku 7 hari (REFRESH_TOKEN_EXPIRES_IN=7d)
- Endpoint POST /api/auth/refresh untuk memperbarui access token
- Endpoint POST /api/auth/logout untuk invalidasi token (blacklist JTI di database)
- Middleware JWT diterapkan secara global di gateway dan per-route di setiap service

Payload JWT berisi: sub (user ID), email, role, jti (JWT ID untuk blacklist), iat, exp.

3.4 Otorisasi OAuth 2.0

Sistem mengimplementasikan Google OAuth 2.0 dengan Authorization Code Flow (bukan Implicit Flow):

1. Client redirect ke GET /api/auth/oauth/google
2. User melakukan consent di halaman Google (accounts.google.com/o/oauth2/v2/auth)
3. Google redirect ke callback dengan authorization code
4. Server menukar code dengan access token Google via POST ke `oauth2.googleapis.com/token`
5. Server mengambil profil user (nama, email, foto) dari Google
6. Upsert user lokal di database dengan flag `oauth_provider='google'`
7. Return JWT access token + refresh token sistem kepada client

3.5 API Gateway

API Gateway diimplementasikan menggunakan Node.js + Express sebagai single entry point dengan fitur:

- JWT Validation — validasi token sebelum request diteruskan ke service tujuan
- Rate Limiting — maksimal 60 request per menit per IP menggunakan `express-rate-limit`
- Routing — meneruskan request ke service yang tepat berdasarkan path prefix
- Header Forwarding — meneruskan `x-user-id` dan `x-user-email` ke downstream service

3.5.1 Peta Routing Gateway

Path Pattern	Service Tujuan	Auth
/api/auth/register, /login, /refresh	auth-service:3001	Public
/api/auth/oauth/google/**	auth-service:3001	Public
/api/auth/logout, /me	auth-service:3001	JWT Required
GET /api/events/**	ticket-service:3002	Public
POST/PUT/DELETE /api/events/**	ticket-service:3002	JWT + Role
/api/tickets/**	ticket-service:3002	JWT Required
GET /api/categories/**	ticket-service:3002	Public
/api/orders/**	payment-service:3003	JWT Required

3.6 Role-Based Access Control

Sistem mengimplementasikan RBAC dengan 3 role:

- user: dapat melihat event, membeli tiket, melihat order dan tiket miliknya
- organizer: dapat membuat, mengubah, dan menghapus event serta kategori tiket
- admin: memiliki akses penuh, hanya dapat diset manual di database

Role disimpan di tabel users pada auth_db dan disertakan dalam JWT payload. RoleMiddleware di ticket-service memverifikasi role sebelum mengizinkan akses ke endpoint yang dibatasi.

BAB 4 STRUKTUR REPOSITORY

4.1 Struktur Folder

Repository GitHub: <https://github.com/anggasspm/uts-pplos-b-2410511080>

```
uts-pplos-b-2410511080/
├── README.md
├── gateway/
│   ├── src/index.js      # Entry point gateway
│   └── .env.example
├── services/
│   ├── auth-service/    # Node.js + Express + MySQL
│   │   ├── src/config/db.js
│   │   ├── src/controllers/ # AuthController, OAuthController
│   │   ├── src/middlewares/ # jwtMiddleware
│   │   ├── src/models/     # AuthModel (UserModel, TokenModel)
│   │   ├── src/routes/     # auth.js
│   │   ├── src/utils/      # jwtHelper
│   │   └── src/index.js
│   ├── ticket-service/  # Laravel 11 + PHP MVC + MySQL
│   │   ├── app/Http/Controllers/
│   │   ├── app/Http/Middleware/
│   │   ├── app/Models/
│   │   └── database/migrations/
│   └── payment-service/ # Node.js + Express + MongoDB
│       └── src/
├── docs/
│   ├── arsitektur.png
│   └── laporan-uts.pdf
├── postman/
│   └── collection.json
└── poster/
    ├── poster-uts.pdf
    └── poster-uts.png
```

4.2 Konvensi Commit

Repository menggunakan format Conventional Commits dengan minimal 15 commit tersebar dalam 7 hari:

- feat: penambahan fitur baru
- fix: perbaikan bug
- docs: perubahan dokumentasi
- refactor: refactoring kode tanpa perubahan fungsional
- chore: perubahan konfigurasi atau setup

Branch yang digunakan: main, feature/jwt, feature/oauth, feature/payment-service, feature/gateway, feature/role-based-access. Setiap branch di-merge ke main melalui Pull Request.

BAB 5 PENGUJIAN

5.1 Alur Pengujian

Pengujian dilakukan menggunakan Postman dengan collection yang telah disediakan di folder /postman. Alur pengujian lengkap:

1. Health check semua service (gateway, auth, ticket, payment)
2. Register sebagai organizer (anggasspm@gmail.com, role: organizer)
3. Login dan simpan access token + refresh token
4. Buat event baru (Konser Dewa 19 — Reunion Tour)
5. Buat kategori tiket VIP, Regular, dan Student
6. Checkout order (POST /api/orders) dengan quantity 2
7. Konfirmasi pembayaran — otomatis generate tiket dengan QR code
8. Lihat tiket (GET /api/tickets) dan validasi QR di pintu masuk
9. Test refresh token dan logout
10. Test role forbidden: create event sebagai user biasa (expected: 403)

5.2 Screenshot

Screenshot lengkap setiap endpoint tersedia di README.md pada folder docs/screenshots/ (28 screenshot total, bernomor 01 hingga 28).

BAB 6 KESIMPULAN

6.1 Kesimpulan

Sistem Pemesanan Tiket Event berbasis microservice berhasil dibangun dengan memenuhi seluruh poin requirement UTS:

- Arsitektur terdiri dari 3 microservice independen (auth, ticket, payment) dan 1 API Gateway sebagai single entry point, masing-masing dengan database terpisah
- REST API menggunakan database relasional MySQL dengan 4 tabel ter-relasi di ticket-service, disertai validasi input, paging, dan filtering
- ticket-service dibangun menggunakan PHP MVC (Laravel 11) dengan pemisahan layer Model, Controller, dan Route yang jelas, serta dikonsumsi oleh payment-service melalui inter-service call
- Autentikasi JWT diimplementasikan dengan access token 15 menit dan refresh token 7 hari, dilengkapi endpoint refresh dan logout dengan blacklist
- Otorisasi OAuth 2.0 dengan Google menggunakan Authorization Code Flow, hasil login dipetakan ke user lokal dengan penyimpanan nama, email, dan foto profil
- API Gateway mengimplementasikan routing per service, validasi JWT, dan rate limiting 60 request per menit per IP

6.2 Saran Pengembangan

- Implementasi Docker dan Docker Compose untuk kemudahan deployment
- Penambahan unit test dan integration test untuk setiap service
- Implementasi message queue (RabbitMQ/Kafka) untuk komunikasi asynchronous antar service
- Penambahan fitur notifikasi email saat tiket berhasil dibuat
- Implementasi caching dengan Redis untuk meningkatkan performa endpoint yang sering diakses